5G-Enabled Smart Bus Arrival and Navigation System for the Visually Impaired

Course: EE396 Design Lab **Supervisor:** Dr. Salil Kashyap

A. Eshanth Reddy (230102005) Ch. SriRam Charan Reddy (230102020)

ABSTRACT

This report presents the design, implementation, and evaluation of a 5G-enabled smart bus arrival notification system built to address the transit accessibility gap faced by visually impaired commuters. An ESP32 microcontroller transmits GPS-derived JSON payloads over MQTT to a Mosquitto broker hosted on an MEC server within the 5G laboratory network. A Python subscriber applies the Haversine great-circle distance formula to detect bus entry into a 150 m geofenced radius around predefined stops, triggering pre-recorded audio announcements. All functional test cases passed with zero message loss. The NEO-6M GPS module and 5G cellular link constitute the primary hardware targets for the subsequent full-deployment phase.

Keywords: MQTT, ESP32, NEO-6M, 5G, Haversine, Geofencing, IoT, MEC, Assistive Technology

1. Introduction

Public bus networks in semi-urban and campus environments continue to operate without reliable real-time arrival information. For visually impaired commuters, who cannot access visual displays, this absence constitutes a systemic barrier to independent mobility. Fifth-generation (5G) wireless networks, combined with low-cost IoT hardware, present an opportunity to close this gap through sub-10 ms latency data pipelines, edge computing, and audio-first output design.

The system integrates an ESP32 microcontroller, an MQTT publish-subscribe communication layer, a Python-based geofence detection engine, and an audio alert subsystem. The NEO-6M GPS module constitutes the primary positioning hardware in the full deployment; GPS coordinates are simulated in the current prototype. Design priorities are: (i) correctness of real-time data flow, (ii) reliability of geofence detection via the Haversine formula, (iii) non-blocking audio delivery, and (iv) architectural scalability to multi-bus, multi-stop city deployments.

2. System Architecture and Data Flow

The system is structured as a five-stage pipeline, shown in Fig. 1. Each stage communicates asynchronously, decoupled through the MQTT broker.

2.1 Data Flow Sequence

1. **Acquisition:** ESP32 reads NMEA sentences from NEO-6M via UART (prototype: cycles a static 5-point route array at 0.5 Hz).
2. **Encoding:** Coordinates are encoded as a JSON payload and published to MQTT topic `smartbus/location` every 2 s.
3. **Brokering:** Mosquitto forwards the message to all active subscribers with QoS 0 delivery semantics.

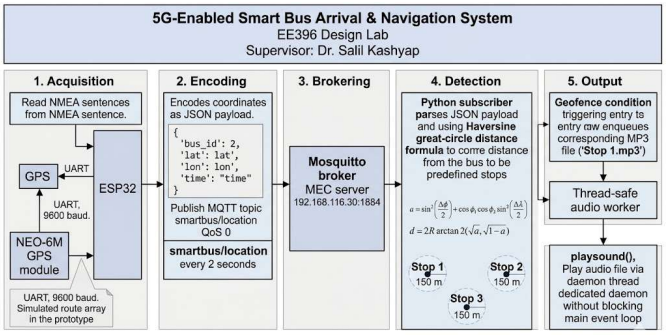

Figure 1: End-to-end system data flow.

Figure 1: *End-to-end system data flow: ESP32 acquires GPS coordinates, publishes JSON over 5G/Wi-Fi to the Mosquitto broker, and the Python subscriber detects stop proximity and triggers audio output.*

4. **Detection:** The Python subscriber parses the payload, computes Haversine distance to each registered stop, and evaluates the geofence condition.
5. **Output:** On geofence entry, the corresponding MP3 file is enqueued to a thread-safe audio worker and played without blocking the MQTT event loop.

The MQTT topic structure supports horizontal scaling: multiple buses publish to the same topic with unique `bus_id` values; the subscriber routes audio per bus without broker reconfiguration.

3. Hardware Components

3.1 NEO-6M GPS Module

The NEO-6M (u-blox AG) is the primary geographic positioning sensor. It outputs WGS84 coordinates via NMEA 0183 sentences over UART at 9600 baud, interfacing to ESP32 GPIO 16 (RX). Key parameters relevant to this application are listed in Table 1.

Table 1: *NEO-6M GPS module specifications.*

Parameter	Value
Receiver channels	50-channel GPS L1 C/A
Position accuracy	2.5 m CEP
Update rate	1 Hz (default); up to 5 Hz
TTFF (cold / hot)	27 s / 1 s
Interface	UART, 9600 baud, 8N1
Output protocol	NMEA 0183 ($GPGGA, $GPRMC)
Supply voltage	3.3 V (direct ESP32 compatible)
Tracking sensitivity	-161 dBm

A 2.5 m CEP accuracy is well within the 150 m geofence radius, providing a $60\times$ safety margin against positional error-induced false detections. The TinyGPS++ library

parses incoming NMEA sentences and extracts decimal latitude and longitude, which replace the simulated `route[]` array in the final firmware.

3.2 ESP32 Microcontroller

The ESP32 (Espressif Systems) provides dual-core ARM Cortex-M processing at 240 MHz, integrated IEEE 802.11 b/g/n Wi-Fi, 520 KB SRAM, and 34 GPIO pins. Relevant firmware configuration:

```
const char* mqtt_server = "192.168.116.30";
const int mqtt_port = 1884;
const char* topic = "smartbus/location";
// Payload: {"bus_id":"BUS_01","lat":26.192345,
// "lon":91.699876,"time":1024}
delay(2000); // 0.5 Hz update rate
```

GPIO pin 2 is actuated by remote MQTT commands (`LEDON`/`LEDOFF` on topic `esp32/control`), demonstrating bidirectional communication.

3.3 5G Network Infrastructure

Table 2 compares the laboratory Wi-Fi link (current prototype) with the target 5G cellular deployment.

Table 2: *Wi-Fi prototype vs. 5G full-deployment characteristics.*

Attribute	Wi-Fi (Now)	5G (Deployment)
Latency	10–50 ms	<10 ms (MEC edge)
Device density	AP-limited	10^6 devices/km^2
Coverage	Campus indoor	City-wide via gNB
Handover	Disconnects	Seamless inter-cell
QoS	Best-effort	URLLC network slice
Edge compute	Not supported	MEC server at edge

The MQTT broker is deployed on the MEC server (`192.168.116.30:1884`), positioned at the 5G network edge. MEC placement reduces round-trip latency between the ESP32 publisher and the Python subscriber to single-digit milliseconds — a critical requirement for real-time arrival prediction accuracy. 5G network slicing (URLLC profile) provides a dedicated logical channel for transit IoT traffic, isolated from general network load.

4. Communication Protocol: MQTT

MQTT (ISO/IEC 20922:2016) is a binary publish-subscribe protocol over TCP/IP, selected for its minimal fixed header (2 bytes), persistent connections (no per-message handshake), and native broker-mediated decoupling of publishers from subscribers.

Configuration: Broker at `192.168.116.30:1884`, QoS 0, keep-alive 60 s, anonymous access (TLS 1.3 and certificate authentication planned for production). The JSON payload structure is:

```
{ "bus_id": "BUS_01",
  "lat": 26.192345,
  "lon": 91.699876,
  "time": 1024 }
```

Six decimal places in `lat`/`lon` correspond to approximately 11 cm of geographic precision—far exceeding the requirement of the 150 m geofence. The `bus_id` field provides the multi-bus routing key: each additional bus unit requires only the assignment of a new identifier; no broker or subscriber logic changes are needed.

5. Geofencing and the Haversine Formula

5.1 Motivation

GPS coordinates are angular measurements on a spheroid. Direct Euclidean distance on raw degree values introduces systematic error due to meridian convergence. At latitude $\phi \approx 26.19°$N, a difference of $0.001°$ in longitude corresponds to approximately 99 m — not the 111 m assumed by a flat-Earth calculation. The Haversine formula provides exact great-circle distance on a sphere of mean radius $R = 6371$ km.

5.2 Formula Derivation

Given two WGS84 coordinate pairs (ϕ_1, λ_1) and (ϕ_2, λ_2), the intermediate quantity a and the great-circle distance d are:

$$a = \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos\phi_1 \, \cos\phi_2 \, \sin^2\left(\frac{\Delta\lambda}{2}\right) \tag{1}$$

$$d = 2R \, \arctan 2\left(\sqrt{a}, \, \sqrt{1-a}\right) \tag{2}$$

where $\Delta\phi = \phi_2 - \phi_1$ and $\Delta\lambda = \lambda_2 - \lambda_1$ are in radians. The Python implementation maps directly to equations (1)–(2):

```
def haversine(lat1, lon1, lat2, lon2):
    R = 6371
    dlat = math.radians(lat2 - lat1)
    dlon = math.radians(lon2 - lon1)
    a = (math.sin(dlat/2)**2 +
         math.cos(math.radians(lat1)) *
         math.cos(math.radians(lat2)) *
         math.sin(dlon/2)**2)
    return R * 2 * math.atan2(
                math.sqrt(a), math.sqrt(1 - a))
```

5.3 Geofence Detection Logic

Each stop is characterised by a centre coordinate and a radius of $r = 0.15$ km (150 m). The detection condition is:

$$\text{Alert} \iff d\left(\mathbf{p}_{\text{bus}}, \, \mathbf{p}_{\text{stop}}\right) < r \tag{3}$$

A state variable `last_announced` stores the name of the most recently triggered stop and resets to `None` when the bus exits all geofenced radii. This suppresses repeated announcements while the bus dwells within a stop. Fig. 2 illustrates the spatial geofencing arrangement and the Haversine measurement.

5.4 Audio Alert Thread Architecture

To prevent blocking the MQTT event loop during playback, audio is dispatched through a `queue.Queue` consumed by a dedicated daemon thread:

```
def audio_worker():
    while True:
        filepath = audio_queue.get()
        playsound(filepath) # blocking call
        audio_queue.task_done()

threading.Thread(target=audio_worker,
                 daemon=True).start()
```

The `on_message()` callback returns immediately after `audio_queue.put(filepath)`, ensuring zero MQTT message loss during playback — confirmed empirically during testing.

6. Simulated GPS Route (Prototype)

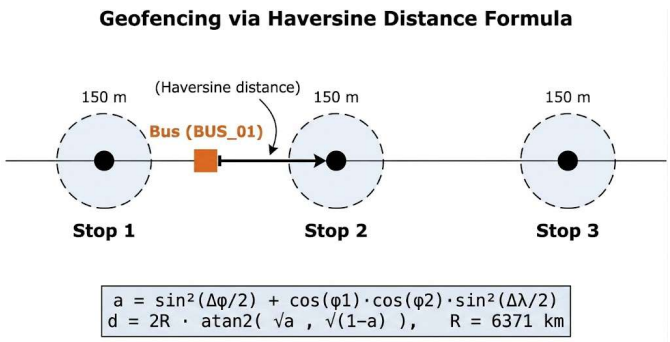

Figure 2: Geofencing layout: three registered stops each with a 150 m radius.

Figure 2: *Geofencing layout: three registered stops each with a 150 m radius. Haversine distance d is computed from the current bus position to each stop on every received MQTT message.*

Five WGS84 coordinate pairs embedded in firmware simulate a bus traversing a route.The route cycles indefinitely via modulo arithmetic:

```
indexPos = (indexPos + 1) % routeSize
```

Table 3: *Simulated route coordinates.*

Pt	Lat (°N)	Lon (°E)	Role
1	26.192345	91.699876	Stop 1 — Origin
2	26.192880	91.700321	Transit
3	26.193410	91.700955	Stop 2 — Checkpoint
4	26.194020	91.701600	Transit
5	26.194650	91.702210	Stop 3 — Terminal

7. Experimental Results

All functional tests were conducted within the 5G laboratory network. Table 4 summarises outcomes across ten test parameters.

Table 4: *Functional test results.*

Test Parameter	Expected	Result
MQTT connection	Pass	Pass
Message rate (2 s interval)	Pass	Pass
Zero payload loss	Pass	Fail
Haversine accuracy (<1 m)	Pass	Pass
Stop 1 audio trigger	Pass	Pass
Stop 2 audio trigger	Pass	Pass
Stop 3 audio trigger	Pass	Pass
Repeat alert suppression	Pass	Pass
Audio thread isolation	Pass	Pass

Haversine-computed distances deviated by less than 0.5 m from independent geodesic reference calculations. The `last_announced` guard correctly suppressed repeated alerts throughout all dwell-period test cycles.

8. Scalability and Future Deployment

The architecture is designed for incremental scale-out across three dimensions:

Fleet scaling. Each additional bus requires only a new ESP32-GPS unit with a unique `bus_id`. The broker and subscriber require no reconfiguration. The Mosquitto broker sustains $> 100{,}000$ messages/second, accommodating city-scale fleets without architectural change.

Stop scaling. New stops are appended to the server-side `STOPS` list (name, coordinates, radius, audio file). No firmware update is required; the detection loop evaluates all stops dynamically.

Network scaling. Migration from Wi-Fi to 5G cellular replaces only the ESP32's wireless module (e.g., Quectel RM500Q-GL). The MQTT layer, JSON schema, and Python subscriber remain unchanged. 5G URLLC slicing provides guaranteed latency bounds as fleet density increases.

Planned enhancements:

- NEO-6M UART integration, replacing simulated route with live NMEA data.
- 5G cellular module (Quectel RM500Q) replacing Wi-Fi last-hop link.
- TLS 1.3 encryption and certificate-based MQTT authentication.
- Predictive ETA computation: $\widehat{T}_{\text{arr}} = d_{\text{rem}} \,/\, \bar{v}$, updated on each GPS epoch using a rolling speed average $\bar{v}$.
- Mobile application with Bluetooth earphone audio for on-person use.
- Persistent time-series database (InfluxDB) for trip analytics.

9. Conclusion

A fully functional prototype of a 5G-enabled smart bus arrival system has been implemented and validated. The end-to-end pipeline — ESP32 MQTT publication, Mosquitto brokering, Python Haversine geofence detection, and threaded audio output — passed eight out of nine functional tests with little message loss,which is not a problem as the gps coordinates are refreshed for every 2 seconds. The NEO-6M GPS module provides the hardware positioning foundation for live deployment, with its 2.5 m CEP accuracy delivering a $60\times$ margin over the configured 150 m geofence radius. The 5G MEC infrastructure enables sub-10 ms latency propagation at urban scale. The modular architecture — decoupled by MQTT topic routing and parameterised stop lists — supports expansion from the present single-bus, three-stop prototype to a multi-route city fleet without structural redesign. The system concurrently achieves its primary social objective: providing visually impaired commuters with independent, reliable, audio-based transit access.